

Data Mining

Block 5 - Pattern discovery & text mining

Daniel Wlazło

SGH Warsaw School of Economics - SMMD-ADA / SMMD-AAB

03 November 2026

Today, hour by hour

- | | |
|--|---------|
| ① Recap - the unsupervised rules of engagement | ~5 min |
| ② Baskets - which products travel together | ~25 min |
| ③ Rules & their three numbers - support, confidence, lift | ~30 min |
| ④ Mining at scale - Apriori & FP-Growth | ~15 min |
| ⑤ Text becomes numbers - tokens, bags, TF-IDF | ~35 min |
| ⑥ Classifying complaints - multiclass, delivered | ~20 min |
| ⑦ Pitfalls - rules on noise, text hygiene | ~15 min |
| ⑧ Lab - notebook 05_pattern_mining_text | ~35 min |

Breaks roughly every 50 minutes.

Last week in five sentences

- 1 Without labels there is no “correct” - only **useful**, argued via internal metrics, stability, and business sense.
- 2 **Distance is a modelling choice**; unscaled features make it for you.
- 3 K-means **always answers**; silhouette and stability tell you how much to believe it.
- 4 Assumptions decide what each tool can see - and **PCA** can only compress redundancy.
- 5 Always ask: *what would this deliverable look like on noise?*

Today: two new kinds of structure - **co-occurrence** and **text** - and a supervised comeback you were promised in Block 1.

The cross-selling question

- New data, same 4,000 customers: which of ten **products** each of them holds - checking, savings, cards, loans, mortgage, insurance, brokerage, deposits.
- The business question: *which products travel together* - and which pairing is a genuine opportunity rather than a coincidence of popularity?
- Uses: cross-sell campaigns, bundling, product-page “customers also hold”, early-warning (a product mix that predicts churn).
- Loaded via `finance_data.load_baskets()` - one row per customer, one 0/1 column per product.

Where else baskets appear

- **Retail** - the literal shopping basket; the field's birthplace and name.
- **Pharmacy & medicine** - co-prescribed drugs, co-occurring diagnoses.
- **Web & apps** - pages visited in one session; features used by one account.
- **Fraud** - devices, addresses and phone numbers shared across applications: co-occurrence as a **ring detector**.

The abstraction is one table: **transactions** × **items**, binary. Whoever can build that table can mine it.

Building the matrix

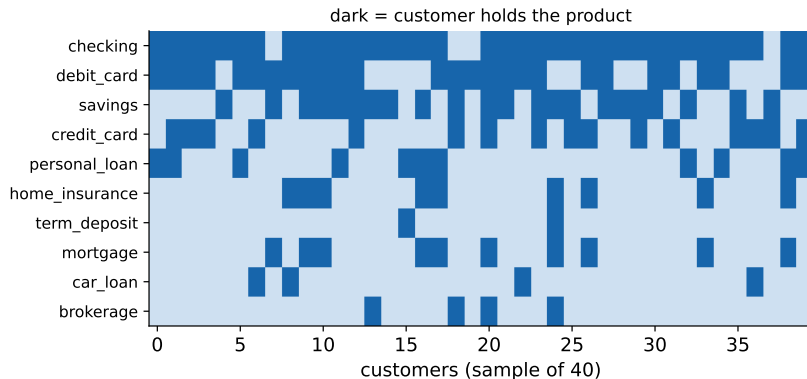
Ours arrives ready (one row per customer, 0/1 per product). In the wild, baskets arrive as *lists* - then:

```
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()
X = te.fit(transactions).transform(transactions)    # lists -> 0/1 matrix
baskets = pd.DataFrame(X, columns=te.columns_)
```

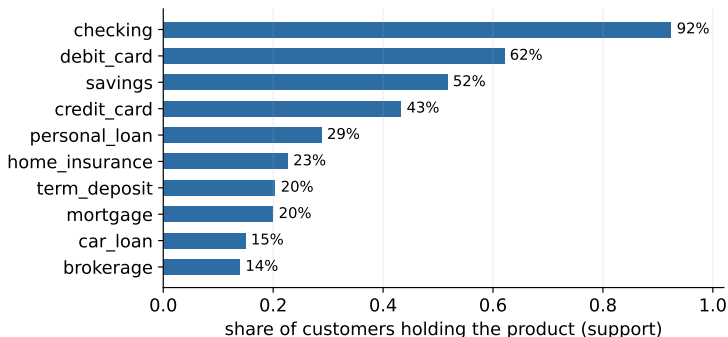
- Long transactional tables (customer, product, date) become baskets with one `pd.crosstab` - but first answer Block 1's question: **what is a row?** One visit? One customer-lifetime? The rules you find inherit the answer.

The basket matrix



- Binary and **sparse** - most customers hold few products. (Retail is sparser still: thousands of items, baskets of five.)
- Every technique today starts from exactly this matrix.

Support: how common is a combination?



- An **itemset** is any set of products; its **support** = share of customers holding all of it:
 $\text{supp}(A) = n_A/n$.
- Single-item supports are just penetration rates. The interesting ones are combinations:
 $\text{supp}(\{\text{mortgage}, \text{insurance}\}) = 17\%$.

From itemsets to rules

- A **rule** $A \Rightarrow B$: customers holding A tend to also hold B . (A : antecedent, B : consequent.)
- The arrow is **our reading direction, not causality** - the data only says the products co-occur; the sale process, not the mortgage, causes the insurance.
- Every itemset spawns many rules ($\{a, b, c\}: a \Rightarrow bc, ab \Rightarrow c, \dots$) - we need numbers to rank them.

Three numbers, next slide - the exam will ask for all three.

Sequences: baskets with a clock (mention)

- Our matrix is a snapshot - *what is held now*. Add timestamps and the question upgrades: **what comes next?** (checking → debit card → first savings → mortgage).
- Sequence mining (PrefixSpan and friends) finds ordered patterns; next-best-product models are its supervised cousins.
- Why banks care: the *order* is the customer lifecycle - and interventions live between the steps.
- Today we stay with snapshots; know that the clocked version exists.

The three numbers of a rule $A \Rightarrow B$

$$\text{support} = \frac{n_{A \cup B}}{n} \quad \text{confidence} = \frac{n_{A \cup B}}{n_A} = P(B | A) \quad \text{lift} = \frac{P(B | A)}{P(B)}$$

- **Support** - how much of the portfolio the rule touches: **materiality**.
- **Confidence** - how reliably A -holders hold B : **precision of the promise**.
- **Lift** - how much more often than *chance*: **is there any relationship at all?** Lift = 1: independence; > 1: attraction; < 1: repulsion.

Mnemonic: support = size, confidence = reliability, lift = surprise.

Worked example, by hand (ours)

Out of $n = 4,000$ customers: $n_{\text{mortgage}} = 800$, $n_{\text{insurance}} = 906$, both = 678.

$$\text{supp} = \frac{678}{4000} = 0.17 \quad \text{conf} = \frac{678}{800} = 0.85 \quad \text{lift} = \frac{0.85}{906/4000} = \frac{0.85}{0.227} \approx 3.7$$

- Read it aloud: *17% of customers hold both; 85% of mortgage holders have the insurance - 3.7× more often than a random customer.*
- Business translation: the bundle works at the mortgage desk. The actionable rule is its **absence**: chase the 15% of mortgage holders *without* insurance.

Your turn: brokerage $\Rightarrow$ term deposit

Out of $n = 4,000$: $n_{\text{brokerage}} = 558$, $n_{\text{deposit}} = 815$, both = 359.

Compute support, confidence, lift. Sixty seconds.

Your turn: brokerage $\Rightarrow$ term deposit

Out of $n = 4,000$: $n_{\text{brokerage}} = 558$, $n_{\text{deposit}} = 815$, both = 359.

Compute support, confidence, lift. Sixty seconds.

$$\text{supp} = \frac{359}{4000} \approx 0.09 \quad \text{conf} = \frac{359}{558} \approx 0.64 \quad \text{lift} = \frac{0.64}{815/4000} \approx 3.2$$

- Weaker promise than the mortgage rule (64% vs 85%) but the same verdict: a real affinity - the **affluent-saver** pattern, which Block 4's segmentation already sketched.

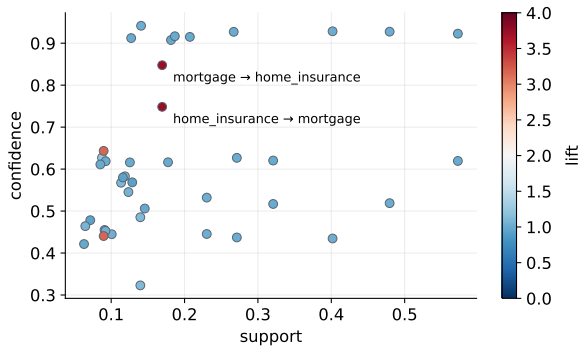
The confidence trap

A rule from our data:

savings $\Rightarrow$ checking confidence = 92.7%

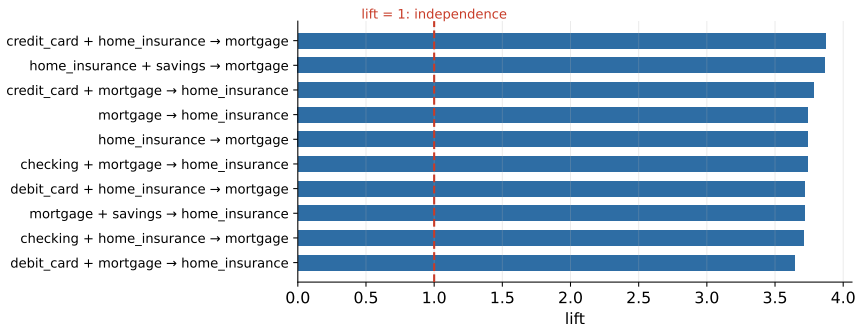
- Sounds spectacular - until you notice 92.4% of everyone has a checking account. Lift = 1.003: pure independence.
- High confidence toward a *popular* consequent is base-rate flattery - Block 2's accuracy trap wearing a basket.
- Rule hygiene: **filter by support** (materiality), **rank by lift** (surprise), **read confidence** for the campaign promise.

The rule landscape



- Every dot is a single-item rule; the two red-hot dots are the mortgage–insurance pair, in both reading directions.
- The crowd at lift ≈ 1 is what most rules look like: popular items co-occurring by **arithmetic**, not affinity.

Top rules by lift - and how to read a rule list



- Notice the **redundancy**: the same mortgage–insurance affinity appears in both directions and re-decorated with irrelevant extras (+ checking, + debit_card).
- One relationship, many rules - deduplicate before reporting, or the deck of “insights” is one insight in ten costumes.

Beyond the big three (mentions)

- **Leverage:** $P(A \cap B) - P(A)P(B)$ - lift's additive sibling; harder to fool with tiny supports.
- **Conviction:** $\frac{1 - \text{supp}(B)}{1 - \text{conf}(A \Rightarrow B)}$ - how much more often the rule would be wrong under independence (= 1: independent; $\rightarrow \infty$: never wrong).
- **Closed / maximal itemsets:** compressed inventories of frequent sets - the cure for rule explosion.
- With p products there are $2^p - 1$ itemsets; at $p = 10$ that's 1,023, at retail scale it's astronomy - **multiple testing is built into the method**. (The sobering demo waits in the pitfalls section.)

From rule to campaign - honestly

- The action hides in the rule's **complement**: 15% of mortgage holders lack the insurance - that is the call list.
- But the rule is co-occurrence, not causality: those 15% may differ *exactly* in whatever made them decline the bundle.
- The honest upgrade: run the campaign as an **experiment** - hold out a random control group, measure uplift, not response.
- And expect **feedback**: acting on a pattern changes it (every mined rule fades as campaigns exploit it) - re-mine on fresh data, date every rule list.

Association rules *nominate* actions; experiments *validate* them.

Why brute force dies

- Counting support for every one of the $2^p - 1$ itemsets - at a supermarket's $p = 30,000$ - means the universe ends first.
- Yet stores mine baskets every night. The rescue is one observation about support:

The Apriori principle (downward closure)

Every subset of a frequent itemset is frequent. Equivalently: if an itemset is infrequent, **every superset is infrequent too** - and can be skipped without counting.

If {brokerage, car_loan} is rare, {brokerage, car_loan, *anything*} is rare. No need to look.

Apriori by hand: one level (exam-friendly)

Five customers, $\text{min_support} = 40\%$ (i.e. ≥ 2 of 5):

1-itemset	supp	candidate 2-itemset	supp
{card}	4/5 ✓	{card, loan}	3/5 ✓
{loan}	3/5 ✓	{card, deposit}	2/5 ✓
{deposit}	2/5 ✓	{loan, deposit}	1/5 ×
{leasing}	1/5 ×	{leasing, <i>anything</i> }: never generated	

- Level 3: only {card, loan, deposit} is even a candidate - and it dies because its subset {loan, deposit} already failed.
- That is the whole algorithm: count a level, prune, build the next level **only from survivors**.

Apriori and FP-Growth

Apriori (1994)

- level-wise: frequent 1-itemsets $\rightarrow$ candidate 2-itemsets $\rightarrow \dots$
- prunes with downward closure at every level
- repeatedly re-scans the data - simple, parallel-friendly, I/O heavy

FP-Growth (2000)

- compresses baskets into a **prefix tree** (FP-tree)
- mines the tree recursively - no candidate generation
- two data scans total; the usual industrial choice

Same definition, same output - only the route differs. `mlxtend` ships both; at our $p = 10$, either is instant.

Practical mining decisions

- **min_support is a business number:** “rules touching $<3\%$ of customers can’t fund a campaign” - set it from materiality, not from what makes the list look rich.
- Too low: rule explosion and noise. Too high: only the obvious survives (checking co-occurs with everything).
- **Long-tail reality:** in retail, most interesting items are individually rare - category-level mining (“any insurance product”) is the standard fix.
- **Output discipline:** filter support $\rightarrow$ rank lift $\rightarrow$ deduplicate mirrors and supersets $\rightarrow$ *then* read.

The other unstructured eighty percent

- Banks are full of text nobody mines: complaints, chat transcripts, advisor notes, application free-text fields.
- Our new corpus: **1,200 customer complaints**, each labelled with one of five categories - card fraud, mortgage, fees, app & service, collections (`finance_data.load_complaints()`).
- Business task waiting at the end: **route each complaint to the right team** - the multiclass classification promised on Block 1's map.
- But models eat numbers. The whole craft of text mining is the bridge: **strings → a feature matrix**.

The bridge, in four steps

- 1 **Normalise** - lowercase, strip punctuation; decisions, not defaults (is “PLN” a word? is “24/7”?).
- 2 **Tokenise** - split into words (or word pieces; or characters).
- 3 **Count** - one column per token, one row per document: the **bag of words**.
- 4 **Weigh** - damp the ubiquitous, boost the distinctive: **TF-IDF**.

Each step throws information away *on purpose* - the art is knowing what you can afford to lose.
(Step 3 loses word order; n-grams buy some of it back.)

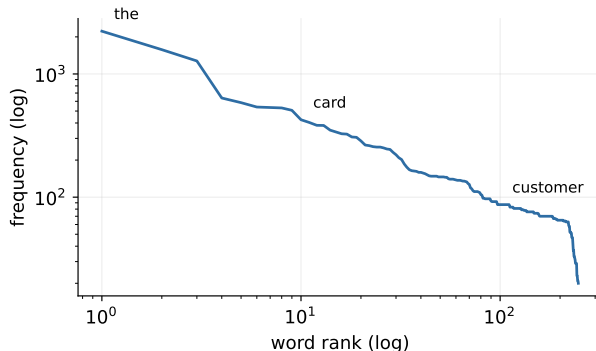
Normalisation decisions: stopwords, stems, lemmas

- **Stopwords** - dropping *the, my, and* shrinks the matrix; dropping **not** reverses meanings (“*not* refunded”). Audit any default list before trusting it.
- **Stemming** - chop endings by rule (*charged, charges, charging* → *charg*); fast, crude, occasionally comic.
- **Lemmatisation** - dictionary-based reduction to base forms; slower, cleaner, and where morphology is rich (Polish: *opłata, opłaty, opłatom, opłatach*) it earns its keep.
- Each choice merges columns of the future matrix - i.e. it is **feature engineering**, with the same rule as always: document the recipe.

Bag of words

- Document $\rightarrow$ vector of token counts; corpus $\rightarrow$ a **documents** $\times$ **vocabulary** matrix - wide, sparse, non-negative. (Structurally: the basket matrix again! words are the items.)
- Lost on purpose: word order. “the bank closed my account” and “my account closed the bank” are the same bag.
- Kept: the lexical fingerprint - which turns out to carry most of the signal for routing-style tasks.
- sklearn: CountVectorizer - a transformer; fit learns the vocabulary, transform counts. (You already know this API.)

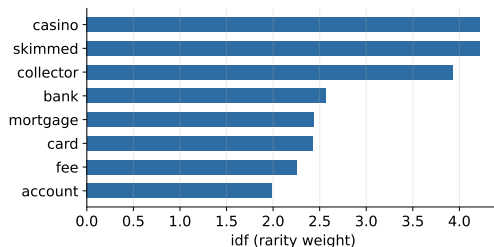
Zipf's law: every corpus looks like this



- A few words are everywhere (the, my, account); most words are rare. Log-log $\Rightarrow$ near-straight line.
- Both ends are trouble: the head carries no distinction, the tail no statistics. `max_df` and `min_df` are the vectoriser's **leash** - prune both ends.

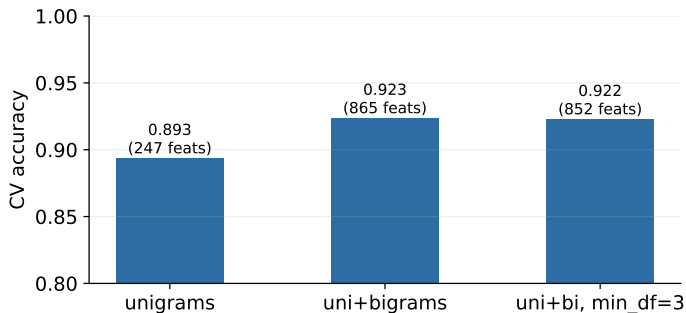
TF-IDF: damp the common, boost the distinctive

$$w_{d,t} = \underbrace{tf_{d,t}}_{\text{count in doc}} \times \underbrace{\left(\ln \frac{1 + n_{\text{docs}}}{1 + n_{\text{docs with } t}} + 1 \right)}_{\text{idf: rarity weight}}$$



- In our corpus: account and bank cost almost nothing; skimmed and casino are expensive - and scream “card fraud”.
- Plus length normalisation, so a rambling complaint doesn’t outweigh a terse one.

N-grams: buying back word order



- Bigrams recover phrases the bag destroys - *collection calls, unauthorized transactions* - worth +3 points of accuracy here.
- The price: the vocabulary explodes (247 → 865 features on 1,200 documents). `min_df` is the leash - our template corpus has a short tail (865 → 852), but real corpora shed *thousands* of one-off tokens with no accuracy loss.

Why $1,200 \times 865$ doesn't hurt: sparsity

- A complaint uses ~ 25 distinct words of the 247-word vocabulary (865 features with bigrams) - the matrix is **$\sim 90\text{--}97\%$ zeros**.
- Sparse storage (CSR) keeps only the non-zeros: megabytes, not gigabytes; a retail-scale corpus (millions $\times$ 100k) still fits.
- Linear models multiply only the stored entries - which is *why* logistic regression on text is fast, and one more reason linear wins wide-and-sparse country.
- Practical corollary: don't `.toarray()` a sparse matrix “just to look” - that is how notebooks die (Block 1's craft, text edition).

Distance between documents: cosine

- Euclidean distance punishes **length**: a long and a short complaint about the same fraud land far apart.
- Cosine similarity compares **direction** - the mix of words, not the amount of them:

$$\cos(\theta) = \frac{x \cdot y}{\|x\| \|y\|}.$$
- This is Block 4's “distance is a modelling choice”, text edition - and the standard choice for TF-IDF vectors.
- Everything distance-based now applies to documents: cluster complaints (k-means on TF-IDF), find near-duplicates, build “similar cases” lookups.

Text is columns now - the whole course applies

Once text is a matrix, Blocks 1–4 come along for free:

Technique	On text becomes	Block
EDA	top words, Zipf, lengths per class	1
Logistic regression	complaint routing (next section)	2
Trees / GBDT	rarely better on sparse text - linear wins	3
Clustering	topic-ish complaint groups	4
PCA (as truncated SVD)	"latent semantic" axes - LSA	4

One craft - representation - unlocks the entire toolbox.

Multiclass, delivered

- Block 1 promised it, today pays: $\text{category} \in \{\text{card_fraud}, \text{mortgage}, \text{fees}, \text{app_service}, \text{collections}\}$ - **five classes**, one label each.
- The pipeline is two familiar pieces:

```
make_pipeline(TfidfVectorizer(), LogisticRegression())
```

- Logistic handles multiclass natively (one score per class, softmax picks); wide-and-sparse is **linear-model country** - thousands of columns, each weakly informative, is exactly what regularised linear models digest best.
- Same golden rule, text edition: the vectoriser is **fitted on training data only** - vocabulary leaks too.

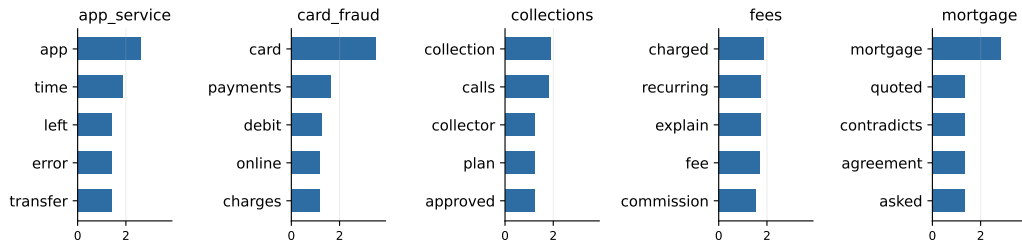
Results: routing at 90%

actual	app_service	65	2	3	2	0
	card_fraud	2	70	0	0	0
	collections	1	5	64	0	2
	fees	2	3	2	61	4
	mortgage	2	1	3	1	65
		app_service	card_fraud	collections	fees	mortgage
		predicted				

- Test accuracy **90%** on five balanced classes (chance: 20%). Errors concentrate where complaints *ramble* across topics - exactly the hard cases a human router also flags.
- Balanced classes make accuracy honest here; real complaint streams are imbalanced - then per-class recall and macro-F1 take over (Block 2–3 reflexes).

What the model reads

strongest words per category (stopwords removed for display)



- Top coefficients per class are a **sanity check you can read**: fraud listens for *card*, *payments*, *online*; collections for *collector*, *calls*; mortgage for *agreement*, *contradicts*.
- An off-topic top word = the leakage smell, text edition (imagine *refunded* predicting fraud - written *after* resolution).

Multiclass metrics in one slide

- **Per-class precision/recall** - five confusion-matrix rows; report the worst, not just the mean (the router's weakest queue is the one that pages you).
- **Macro-F1**: average F1 over classes, each class equal - the imbalance-honest default.
- **Weighted-F1 / accuracy**: dominated by big classes - fine only when classes are balanced (ours are, by construction).
- Thresholds exist here too: route automatically only above a confidence bar, send the rest to a **human queue** - the expected-cost logic of Block 3, multiclass edition.

From accuracy to operations: the confidence bar

Route automatically only when the model is sure; queue the rest for humans. Our router, on the test set:

confidence bar	auto-routed	accuracy (auto)	human queue
none	100%	90.3%	0%
0.5	91%	93.6%	9%
0.6	81%	98.3%	19%
0.7	79%	100%	21%

- The bar is Block 3's threshold logic, multiclass edition: trade automation volume against error cost, with the business.
- At a 0.6 bar: four in five complaints route themselves, nearly error-free - and the humans see only the genuinely hard fifth.

Naive Bayes: the classical text baseline (mention)

- Counts word frequencies per class, multiplies probabilities as if words were independent - **naively**; hence the name.
- Wrong assumption, strong baseline: fast, tiny, decent on small text data - for decades *the* spam-filter algorithm.
- sklearn: MultinomialNB; one line to swap into our pipeline (notebook exercise - and the baseline discipline says you should).

Embeddings & LLMs: the modern bridge (where it fits)

- TF-IDF sees *words*; embeddings see *meaning* - “the app crashes” and “application keeps freezing” land close in embedding space, far in TF-IDF space.
- Modern practice: an embedding model (or LLM) turns text into dense vectors → the same classical models we already know. **Block 2’s “LLMs as feature factories”**, delivered.
- TF-IDF still wins when: data is small, latency and cost matter, and the **auditor asks why** - coefficients over vibes.
- Rule of the shop: start with TF-IDF + logistic; let embeddings *earn* their complexity (the Block 3 discipline, verbatim).

The sobering demo, basket edition

We shuffled every product column independently - destroying all co-occurrence, keeping every penetration rate. Then mined with the same settings:

	real baskets	shuffled baskets
rules passing the same filters	545	312
best lift	4.28	1.14

- The rule *count* is nearly meaningless - the same filters pass **three hundred “rules”** on pure noise.
- What noise cannot fake is **lift**: 1.14 versus 4.28. Rank by lift, and demand it clears noise levels - Block 4's noise question, answered with a number.

Text hygiene

- **Vectoriser inside the pipeline** - vocabulary and idf are fitted parameters; fitting them on all data is leakage (rare-word idf especially).
- **Vocabulary drifts** - new products, new slang, new fraud patterns; a 2024 vocabulary reads 2026 complaints badly. Out-of-time testing applies to text too.
- **PII everywhere** - names, account numbers, phone numbers live inside free text; anonymise before mining, log nothing raw (GDPR is not a footnote).
- **Language reality** - our corpus is conveniently English; Polish morphology (*opłata*, *opłaty*, *opłatom*, *opłatach*) makes lemmatisation earn its keep.

A word on Polish (and any inflected language)

- Our corpus is conveniently English. Polish text multiplies every noun into a dozen surface forms - bag-of-words sees *opłata* and *opłatach* as strangers.
- Survival kit: lemmatisation with a Polish model (spaCy `pl_core_news`, Stanza, Morfeusz), diacritics kept (*łata* $\neq$ *lata*), and **character n-grams** as the robust fallback - they shrug at inflection and typos.
- Embedding models are trained multilingually and largely dissolve the problem - one more genuine LLM-era win (previous slide's trade-offs still apply).
- Project note: mining Polish text is welcome - state your normalisation recipe explicitly.

Common sins of both crafts

- 1 Reporting **confidence without lift** - base-rate flattery as insight.
- 2 **One insight in ten costumes** - mirror and superset rules padding a slide deck.
- 3 Reading rules **causally** - and designing an intervention that changes the pattern it relied on.
- 4 **Stopword cargo-culting** - deleting words like *not* (“not refunded”!) because a default list said so.
- 5 **Evaluating text models on their own templates** - near-duplicate documents across the split; the duplicate leak, text edition.
- 6 Shipping a router with **no human queue** - 90% right means one complaint in ten lands on the wrong desk, silently.

Laboratory

Notebook 05_pattern_mining_text - laptops out.

Lab today

Notebook 05_pattern_mining_text

- mine the product baskets: Apriori, rules, the lift-vs-confidence lesson
- run the shuffled-basket sobering check yourselves
- build the text bridge: counts $\rightarrow$ TF-IDF $\rightarrow$ n-grams
- train and audit the complaint router

Done =

- three rules you would act on, with all three numbers and one honest sentence each
- a router whose confusion matrix you can *explain*

Optional extras

- Naive Bayes bake-off
- cluster the complaints (Block 4 meets Block 5)

Summary

What to take home from Block 5.

Block 5 in five sentences

- 1 Baskets are one abstraction - **transactions** × **items** - and it fits far more than shopping.
- 2 A rule has three numbers: **support** for materiality, **confidence** for the promise, **lift** for whether anything is there at all.
- 3 **Apriori's** downward closure is why mining terminates before the universe does.
- 4 Text becomes columns via **bag-of-words and TF-IDF** - and then the whole course applies, including the golden rule (vocabulary leaks too).
- 5 Filters pass rules on noise and routers misfile one in ten - **honesty clauses ship with the deliverable**.

Project status & reminder - two weeks to go

- Two weeks to the presentation - and remember the upload rule: **all code + the final presentation, uploaded before Block 7 starts**. Plan the polish time now, not on the last evening.
- This week's target: interpretation and a written recommendation; start the presentation skeleton. The presentation & defence is **20% of the project grade** - budget real time for it.
- Text or baskets in your project? The honesty kit extends: lift over confidence, deduplicated rules, vectoriser in the pipeline, and one out-of-vocabulary sentence in the limitations.
- The rubric rewards *correct and interpreted* over *complex and mute* - today gave you two more interpretable tools, use them that way.

Exam practice - multiple choice

Q1. The rule savings $\Rightarrow$ checking has confidence 92.7% and lift 1.00. Checking's overall penetration is 92.4%. Act on the rule?

- ☐ a) yes - 92.7% confidence is outstanding
- ☐ b) no - lift ≈ 1 means savings tells us nothing beyond the base rate
- ☐ c) yes, but only for new customers
- ☐ d) cannot decide without support

Q2. In TF-IDF, the idf factor:

- ☐ a) removes stop words from the vocabulary
- ☐ b) counts each word's frequency inside one document
- ☐ c) down-weights words appearing in many documents, up-weights distinctive ones
- ☐ d) normalises document length

Exam practice - open question

In the style of the term-0 exam (~60% open questions)

Your complaint router ships at 90% accuracy. Six months later the bank launches a new product, and complaints about it start arriving. Describe what happens to the router - loudly or silently? - and design the minimal monitoring that would catch it.

4-6 sentences. "Retrain sometimes" is not yet a design.

Next week: neural networks & responsible ML.

The last new model family - and the questions that outlast every model.

Keep learning - Block 5 reading

- **Agrawal & Srikant** - *Fast Algorithms for Mining Association Rules* (VLDB 1994) - the original Apriori paper, still readable.
- **Sebastiani** - *Machine Learning in Automated Text Categorization* (ACM Computing Surveys, 2002) - the classic survey behind today's router.
- **scikit-learn User Guide** - *Working with text data* tutorial: our router pipeline, step by step.
- **mlxtend documentation** - the `frequent_patterns` module: everything Apriori beyond today's lab.